

SUPSI

Proxy basato su Appwrite

Studente/i

**Mazza Luca
Mondini Manuela**

Relatore

Brocco Amos

Correlatore

-

Committente

Brocco Amos

Corso di laurea

**Ingegneria informatica
(Informatica TP)**

Modulo

C11160

Anno

2025 / 2026

Data

15 marzo 2026

Indice

1 Abstract	1
2 Introduzione	3
2.1 Pianificazione	4
2.2 Requisiti	6
3 Stato dell'arte	7
4 Design e implementazione	8
4.1 API	8
5 Test e validazione	9
5.1 Risultati	9
6 Risultati	10
6.1 Manuale d'uso	10
7 Conclusioni	11
7.1 Sviluppi futuri	11
Glossario	I
Sitografia	III

Elenco delle figure

Elenco delle tabelle

2.1	Tecnologie utilizzate nel progetto	4
2.2	Requisiti di Appcomm	6

Capitolo 1

Abstract

Le policy di sicurezza di rete moderne spesso isolano sia client che server rendendo impraticabili le connessioni dirette senza l'utilizzo di complesse configurazioni. Questo progetto espone il design e lo sviluppo di un'architettura client-server, pensata per superare le limitazioni imposte da firewall e NAT nelle reti private. Il meccanismo centrale sostituisce le tradizionali connessioni socket con un approccio basato su REST + WebSocket, utilizzando Appwrite¹ [1] come Proxy. Il sistema crea un canale di comunicazione sicuro su cui client e server interagiscono asincronamente tramite eventi di creazione di documenti nel database di Appwrite.

Viene quindi sviluppata una libreria modulare in C++ (con QT [2]) per astrarre l'API di Appwrite, implementando funzionalità come la gestione delle sessioni, l'identificazione dei canali basata su UUID e la serializzazione dei payload in JSON. L'implementazione di tale libreria viene validata attraverso lo sviluppo di una semplice applicazione Proof of Concept (PoC), un frontend, realizzato con Qt/QML e compilato con WebAssembly [3] per l'esecuzione in browser, e un servizio backend nativo in C++.

Modern security policies often restrict direct client-server communication, especially in private networks. This project details the design and development of a client-server architecture, designed to overcome the limitations of firewalls and NAT in private networks. The core mechanism replaces the traditional client-server socket connections with a REST + WebSocket based approach, using Appwrite as a Proxy. The system creates a secure communication channel on which clients and servers interact asynchronously via document creation events in the Appwrite-hosted database.

A modular C++ (with QT) library is developed to abstract the Appwrite API, implementing features such as session management, UUID-based channel identification and JSON payload serialization. The implementation of said library is validated through the development of a simple Proof of Concept (PoC) application, a frontend, built with Qt/QML and compiled with WebAssembly for browser-based execution, and a native C++ backend service.

¹<https://appwrite.io>

Capitolo 2

Introduzione

Il progetto nasce dall'esigenza di risolvere una delle sfide che più comunemente affligge le architetture distribuite moderne, ovvero la comunicazione bidirezionale asincrona tra un client ed un server che risiedono in reti private.

Lo scopo del progetto è quindi lo sviluppo di una libreria C++ che permetta di utilizzare Appwrite come middleware (proxy) per facilitare la creazione di canali di comunicazione sicuri e scalabili tra client e server. Al posto di tentare connessioni dirette instabili o poco sicure, entrambe le entità si connettono ad un'istanza Appwrite che funge da intermediario autenticato per lo scambio di messaggi strutturati in tempo reale. Il sistema deve garantire la sicurezza, fornendo accesso solamente ad utenti autenticati e autorizzati per l'accesso ai canali di comunicazione, delegando la gestione di credenziali, sicurezza e persistenza all'infrastruttura sottostante di Appwrite.

Gli obiettivi principali del lavoro sono articolati in diverse aree chiave:

- **Progettazione di un'architettura sicura:** è necessario definire un modello di comunicazione dove il server agisca da amministratore dell'infrastruttura di comunicazione (essendo autenticato tramite API Key) ed il frontend come utente finale (autenticandosi come utente "normale"). Il server è inoltre responsabile della gestione dei canali di comunicazione, fornendo un meccanismo di creazione, lettura, aggiornamento e cancellazione (CRUD) per canali, sessioni, mentre invece il frontend si preoccupa semplicemente di connettersi a canali esistenti, inviare messaggi e ricevere notifiche di nuovi messaggi.
- **Sviluppo della libreria appcomm:** una libreria modulare, scritta in C++ e basata su Qt, che fornisca un'interfaccia semplice ed intuitiva per interagire con l'API di Appwrite è necessaria per la strutturazione del progetto; questa deve includere funzionalità per la gestione delle sessioni, l'identificazione dei canali basandosi su UUID, la serializzazione, validazione e impacchettamento dei payload e la gestione degli eventi di comunicazione in modo asincrono.
- **Proof of Concept:** per validare l'efficacia e il funzionamento corretto del sistema è necessario lo sviluppo di una semplice applicazione che dimostri la maniera di utilizzo della libreria in un contesto reale ed il funzionamento tecnico del meccanismo di comunicazione. Questa consiste in una chat semplice, dove un frontend, costruito con Qt/QML e compilato con WebAssembly per l'esecuzione tramite web browser, comunica con un servizio backend nativo in C++, attraverso i canali di comunicazione gestiti da Appwrite tramite appcomm. Per semplificare la dimostrazione, il backend si limita al ruolo di *echo server*.
- **Bootstrap deploy:** per facilitare l'adozione dell'intermediario Appwrite in soluzioni esistenti, è necessario che, in presenza di un'istanza "pulita" di Appwrite ed il server, quest'ultimo sia in grado, solamente tramite la chiave API e l'URL dell'istanza, di creare in autonomia l'infrastruttura necessaria per il funzionamento, dimodoché sia possibile integrare Appwrite sia come "primo cambiamento" in un'architettura esistente, sia nel caso si necessiti di scalare orizzon-

talmente un sistema già configurato con questo proxy, permettendo di cambiare istanza in modo dinamico, senza necessità di riconfigurazione.

Il prodotto si avvale di uno stack tecnologico moderno e cross-platform, con l'obiettivo di massimizzare la portabilità e l'integrazione in diversi contesti applicativi, con un focus particolare su soluzioni web-based grazie alla compilazione in WebAssembly.

Tecnologia	Ruolo
Appwrite	Funge da middleware (proxy) per la comunicazione tra client e server
C++	Linguaggio di programmazione per la libreria e il backend
Qt	Framework per lo sviluppo della libreria e del frontend
QML	Linguaggio per la definizione dell'interfaccia utente del frontend
WebAssembly	Compilazione del frontend per l'esecuzione in browser
JSON	Formato per la serializzazione dei messaggi scambiati tra client e server
Docker	Contentitore per il deploy dell'istanza Appwrite e del backend

Tabella 2.1: Tecnologie utilizzate nel progetto

2.1 Pianificazione

Di seguito sono descritte le fasi principali che sono state pianificate per lo sviluppo del progetto. Il piano non è ovviamente rigido, data la natura iterativa dello sviluppo software, ma fornisce una roadmap generale per l'implementazione e la validazione del sistema.

La presenza di un piano che non sia troppo statico è fondamentale per valorizzare l'obiettivo di sviluppo piuttosto che la semplice realizzazione di un prodotto, permettendo di adeguarsi dinamicamente alle esigenze che potenzialmente emergono durante le varie fasi di sviluppo e di discussione con il relatore.

Essendo anche le tecnologie usate relativamente nuove per tutti i membri del team, soprattutto per quanto riguarda Appwrite e WebAssembly, è stato necessario prevedere una fase iniziale di analisi e studio approfondito, prima di procedere con l'implementazione vera e propria, per garantire una comprensione solida dei meccanismi da implementare.

1. **Analisi:** occorre dello studio dedicato ad approfondire come possa essere utilizzato Appwrite come intermediario per la comunicazione tra client e server, comprendere come adattare l'API dello stesso alle esigenze tecniche del progetto e studiare le tecnologie coinvolte;
2. **Pianificazione:** è necessario definire un piano di sviluppo per quanto riguarda la libreria appcomm, identificando le funzionalità chiave da implementare, formalizzandole tramite delle task. Questo va ripetuto anche per la parte di sviluppo del frontend e del backend;
3. **Design:** è necessario definire un'architettura per la libreria appcomm che sia modulare, scalabile e manutenibile, identificando i moduli chiave, le loro responsabilità e le interazioni tra di essi. Inoltre, è necessario definire un'architettura generale per il sistema, identificando i componenti chiave, le loro responsabilità e le interazioni tra di essi. Ciò include la produzione di diversi diagrammi che al meglio esplichino le scelte architettureali, come ad esempio diagrammi delle classi, diagrammi di sequenza e diagrammi di flusso;
4. **Implementazione Libreria:** occorre implementare dapprima la libreria appcomm, la quale verrà implementata con il classico approccio per task, dove si analizza dapprima il relativo requisito, si implementa quanto necessario per soddisfarlo, si definisce un'unità di test per validarne il funzionamento e si procede con quanto segue. Lo sviluppo della libreria deve

essere imperativamente preceduto da una fase di design dell'architettura della stessa, per garantire una struttura modulare, scalabile e manutenibile;

5. **Implementazione PoC:** una volta che la libreria è sufficientemente matura, è possibile procedere con lo sviluppo del Proof of Concept, il quale ha l'unico scopo di validare il funzionamento tecnico del sistema, deve quindi essere il più semplice possibile, limitato appunto alla dimostrazione del funzionamento della libreria;
6. **Test e Validazione:** una volta che il PoC è completo, è necessario procedere con una fase di test e validazione del prodotto finito, per garantire che tutte le funzionalità siano implementate correttamente e che il sistema funzioni come previsto, dal punto di vista del client e da quello del server, in scenari di comunicazione reali;
7. **Documentazione:** parallelamente alle fasi di sviluppo, è necessario procedere con la stesura della documentazione tecnica del progetto, che include sia la documentazione della libreria appcomm, sia un manuale d'uso per il prodotto finito, che spieghi come utilizzare il sistema, come configurarlo e come integrarlo in soluzioni esistenti. Questa guida utente riassume la documentazione tecnica integrata nei vari moduli della libreria, fornendo esempi pratici di utilizzo e consigli di troubleshooting per facilitare l'adozione del sistema.

Questo rapporto è stato scritto durante tutta la durata del progetto, parallelamente a tutte le fasi descritte sopra.

2.2 Requisiti

ID	Descrizione	Note
R-01	<i>Appcomm</i> deve astrarre l'API di Appwrite	Modulo dedicato
R-02	<i>Appcomm</i> deve esporre lo stato di connessione con Appwrite	-
R-03	<i>Appcomm</i> detiene dei modelli di dominio	-
R-04	<i>Appcomm</i> deve filtrare i messaggi sulla base del destinatario	-
R-05	<i>Appcomm</i> necessita un sistema di Recovery per i messaggi persi	-
R-06	<i>Appcomm</i> tiene una cache degli ultimi n messaggi	-
R-07	<i>Appcomm</i> immagazzina una policy di persistenza dei messaggi	-
R-08	<i>Appcomm</i> gestisce un modello unificato per gli errori	-
R-09	<i>Appcomm</i> rende intercambiabile la modalità di comunicazione (RT/Poll)	-
R-10	<i>Appcomm</i> fornisce un Wrapper per la gestione di connessione/eventi	-

Tabella 2.2: Requisiti di Appcomm

Capitolo 3

Stato dell'arte

Capitolo 4

Design e implementazione

4.1 API

Di seguito è riportato un esempio di codice per l'inizializzazione.

```
// Esempio di inizializzazione  
float x = 0.5f;  
std::vector<float> input = {x, x, x}; // Input per il modello  
// Altri parametri...
```

Capitolo 5

Test e validazione

5.1 Risultati

Capitolo 6

Risultati

6.1 Manuale d'uso

Capitolo 7

Conclusioni

7.1 Sviluppi futuri

Glossario

API Application Programming Interface. Un insieme di definizioni e protocolli utilizzati per costruire e integrare il software applicativo. 1

Appwrite Una piattaforma open-source per lo sviluppo di applicazioni web e mobile, che fornisce una serie di servizi backend come autenticazione, database, storage e funzioni serverless. 4

JSON JavaScript Object Notation. Un formato di testo standard aperto, leggero e leggibile dall'uomo, utilizzato per lo scambio di dati tra client e server. 1

NAT Network Address Translation. Una tecnica usata dai router per modificare gli indirizzi IP dei pacchetti in transito, permettendo a più dispositivi di condividere un singolo indirizzo IP pubblico. 1

Proxy Un componente software o hardware che funge da intermediario tra un client e un server, inoltrando le richieste e le relative risposte. 1

UUID Universally Unique Identifier. Un identificativo standardizzato a 128 bit utilizzato nei sistemi informatici per identificare in modo univoco informazioni e risorse. 1

WebAssembly Un formato di istruzioni binario portabile, progettato per consentire l'esecuzione di codice ad alte prestazioni all'interno dei browser web. 1, 4

Sitografia

- [1] Appwrite Team. *Appwrite Documentation - Open-Source Backend Server*. 2026. URL: <https://appwrite.io/docs> (visitato il 18/02/2026).
- [2] The Qt Company. *Qt 6 Documentation*. 2026. URL: <https://doc.qt.io/qt-6/> (visitato il 18/02/2026).
- [3] WebAssembly Community Group. *WebAssembly Specification*. 2026. URL: <https://webassembly.org/> (visitato il 18/02/2026).